

Software Engineering, 10th Global Edition — Chapter 2: Software Processes

Sections 2.1 (Software Process Models) and 2.2 (Process Activities) — Study Questions

Batch 1 – Questions 1 to 20: Section 2.1 Software Process Models

Q1. A software process model (or Software Development Life Cycle, SDLC model) is best described as:

- a) A simplified representation of a software process
- b) A complete and exhaustive description of every process activity
- c) A programming language
- d) A testing tool only

Answer: a) A simplified representation of a software process

Q2. Why does each process model only provide partial information about a process?

- a) Because models are randomly generated
- b) Each model represents the process from a particular perspective, e.g., an activity model may not show the roles of people involved
- c) Because process models only apply to hardware
- d) Because process models are always outdated

Answer: b) Each model represents the process from a particular perspective, e.g., an activity model may not show the roles of people involved

Q3. Which three general process models are covered in Section 2.1?

- a) Spiral, V-model, and RUP only
- b) The waterfall model, incremental development, and integration and configuration
- c) Agile, Scrum, and Kanban
- d) Object-oriented, structured, and functional design

Answer: b) The waterfall model, incremental development, and integration and configuration

Q4. What does the waterfall model do with the fundamental process activities of specification, development, validation, and evolution?

- a) Represents them as separate, sequential process phases
- b) Runs them all in parallel
- c) Interleaves them continuously
- d) Removes validation entirely

Answer: a) Represents them as separate, sequential process phases

Q5. What does incremental development do with specification, development, and validation activities?

- a) Keeps them strictly separate and sequential
- b) Skips validation until the very end
- c) Interleaves them, developing the system as a series of increasingly functional versions
- d) Performs them only once at the start

Answer: c) Interleaves them, developing the system as a series of increasingly functional versions

Q6. What does the integration and configuration approach rely on?

- a) Avoiding all software reuse
- b) A strictly sequential waterfall process
- c) The availability of reusable components or systems
- d) Writing every component from scratch

Answer: c) The availability of reusable components or systems

Q7. According to the text, what determines the right process model for a project?

- a) Only the programming language chosen
- b) Only the preference of the lead programmer
- c) The customer and regulatory requirements, the environment where the software will be used, and the type of software being developed
- d) Only the size of the development team

Answer: c) The customer and regulatory requirements, the environment where the software will be used, and the type of software being developed

Q8. Why is safety-critical software usually developed using a waterfall process?

- a) Because safety-critical software never changes
- b) Because it is the cheapest approach
- c) Because waterfall requires no documentation
- d) Because lots of analysis and documentation is required before implementation begins

Answer: d) Because lots of analysis and documentation is required before implementation begins

Q9. According to the text, how are business systems increasingly being developed?

- a) By configuring existing systems and integrating them to create a new system with the required functionality
- b) Always from scratch using the waterfall model
- c) Without any customer involvement
- d) Only through formal mathematical methods

Answer: a) By configuring existing systems and integrating them to create a new system with the required functionality

Q10. What does the Rational Unified Process (RUP) do?

- a) Applies only to hardware engineering
- b) Brings together elements of all the general process models and supports prototyping and incremental delivery
- c) Replaces the need for any process model
- d) Is identical to the waterfall model

Answer: b) Brings together elements of all the general process models and supports prototyping and incremental delivery

Q11. The RUP is normally described from which three perspectives?

- a) A hardware, software, and network perspective
- b) A testing, coding, and design perspective only
- c) A dynamic perspective, a static perspective, and a practice perspective
- d) A financial, legal, and marketing perspective

Answer: c) A dynamic perspective, a static perspective, and a practice perspective

Q12. What are the four phases of the RUP?

- a) Requirements, coding, debugging, and release
- b) Inception, elaboration, construction, and transition
- c) Planning, analysis, design, and testing
- d) Discovery, prototyping, scaling, and archiving

Answer: b) Inception, elaboration, construction, and transition

Q13. The first published model of the software development process was derived from:

- a) Engineering process models used in large military systems engineering

- b) Object-oriented design principles
- c) Internet software engineering practices
- d) Agile manifesto principles

Answer: a) Engineering process models used in large military systems engineering

Q14. Why is the waterfall model also known as the 'software life cycle'?

- a) Because every phase runs simultaneously
- b) Because it repeats infinitely
- c) Because of the cascade from one phase to another
- d) Because it was first used for websites

Answer: c) Because of the cascade from one phase to another

Q15. The waterfall model is described as an example of a:

- a) Risk-driven process
- b) Reuse-driven process
- c) Test-driven process
- d) Plan-driven process

Answer: d) Plan-driven process

Q16. What are the five stages of the waterfall model, in order?

- a) Specification; prototyping; refactoring; release; archiving
- b) Design; coding; marketing; sales; support
- c) Requirements analysis and definition; system and software design; implementation and unit testing; integration and system testing; operation and maintenance
- d) Coding; testing; debugging; deployment; retirement

Answer: c) Requirements analysis and definition; system and software design; implementation and unit testing; integration and system testing; operation and maintenance

Q17. In the waterfall model, what is normally the longest life-cycle phase?

- a) Operation and maintenance
- b) Implementation and unit testing
- c) Requirements analysis and definition
- d) Integration and system testing

Answer: a) Operation and maintenance

Q18. Why do customers and developers sometimes prematurely freeze the software specification in a waterfall process?

- a) Because it reduces testing time to zero
- b) Because freezing the specification is required by law
- c) Because changing documents at later stages requires customer approval and delays the overall process
- d) Because requirements never change in practice

Answer: c) Because changing documents at later stages requires customer approval and delays the overall process

Q19. According to the text, for which type of systems is the waterfall model appropriate? (choose the best single category)

- a) Any system with changing requirements
- b) Systems developed informally by small teams
- c) Systems with no need for documentation
- d) Embedded systems where the software has to interface with hardware systems

Answer: d) Embedded systems where the software has to interface with hardware systems

Q20. What is 'formal system development', mentioned as a variant of the waterfall model?

- a) A method that skips specification entirely
- b) An approach where a mathematical model of a system specification is created and refined using mathematical transformations into executable code
- c) A purely visual, diagram-only design approach
- d) An informal agile technique with no documentation

Answer: b) An approach where a mathematical model of a system specification is created and refined using mathematical transformations into executable code

Batch 2 – Questions 21 to 36: Incremental Development and Integration & Configuration

Q21. Incremental development is based on the idea of:

- a) Developing an initial implementation, getting feedback, and evolving the software through several versions
- b) Testing the system only after full development is complete
- c) Delivering only one final version with no feedback
- d) Writing complete documentation before any code is written

Answer: a) Developing an initial implementation, getting feedback, and evolving the software through several versions

Q22. In a plan-driven approach to incremental development, when are the system increments identified?

- a) In advance
- b) Randomly during testing
- c) Only after the system is fully delivered
- d) They are never identified

Answer: a) In advance

Q23. In an agile approach to incremental development, how are later increments decided?

- a) Early increments are identified, but later increments depend on progress and customer priorities
- b) They are decided purely by a random lottery
- c) Later increments are decided by a different company
- d) All increments are fixed in advance and never change

Answer: a) Early increments are identified, but later increments depend on progress and customer priorities

Q24. Incremental software development is better than a waterfall approach for systems:

- a) Whose requirements are likely to change during the development process
- b) That have no functionality
- c) That require no customer feedback
- d) That will never be modified after release

Answer: a) Whose requirements are likely to change during the development process

Q25. Which of the following is listed as a major advantage of incremental development over the waterfall model?

- a) It removes the need for a development team
- b) It eliminates the need for testing
- c) The cost of implementing requirements changes is reduced
- d) It guarantees zero bugs

Answer: c) The cost of implementing requirements changes is reduced

Q26. Why is it easier to get customer feedback with incremental development, according to the text?

- a) Documentation replaces the need for any demonstration
- b) Feedback is collected only once a year
- c) Customers are not allowed to see the software until release
- d) Customers can comment on demonstrations of working software, which is easier to judge than design documents

Answer: d) Customers can comment on demonstrations of working software, which is easier to judge than design documents

Q27. From a management perspective, what is one problem with the incremental approach?

- a) It produces too much documentation
- b) The process is not very visible, making it harder for managers to measure progress with regular deliverables
- c) It requires no planning at all
- d) It cannot be used for any business system

Answer: b) The process is not very visible, making it harder for managers to measure progress with regular deliverables

Q28. What tends to happen to system structure as new increments are added?

- a) It automatically becomes cleaner
- b) It becomes impossible to test
- c) It stays exactly the same forever
- d) It tends to degrade, becoming messier as new functionality is added

Answer: d) It tends to degrade, becoming messier as new functionality is added

Q29. What do agile methods suggest to reduce structural degradation from incremental development?

- a) Avoid testing to save time
- b) Regularly refactor (improve and restructure) the software
- c) Never modify the code after it is written
- d) Stop adding new increments after the second one

Answer: b) Regularly refactor (improve and restructure) the software

Q30. For large, complex, long-lifetime systems developed by different teams, what does incremental development's problems require?

- a) A single developer working alone
- b) No planning at all
- c) Avoiding any architecture altogether
- d) A stable framework or architecture planned in advance, with clearly defined team responsibilities

Answer: d) A stable framework or architecture planned in advance, with clearly defined team responsibilities

Q31. What is the key difference between incremental development and incremental delivery?

- a) Incremental delivery means no feedback is ever collected
- b) Incremental development does not necessarily mean delivering each increment to the customer, whereas incremental delivery means the software is used in real operational processes
- c) Incremental development always skips validation
- d) They are exactly the same concept

Answer: b) Incremental development does not necessarily mean delivering each increment to the customer, whereas incremental delivery means the software is used in real operational processes

Q32. What do reuse-oriented (integration and configuration) approaches rely on?

- a) A purely sequential waterfall structure
- b) Avoiding all third-party software
- c) A base of reusable software components and an integrating framework for composing them
- d) Writing every line of code from scratch

Answer: c) A base of reusable software components and an integrating framework for composing them

Q33. Which of the following is listed as a type of software component frequently reused?

- a) Marketing brochures
- b) Physical hardware casings
- c) Printed user manuals
- d) Stand-alone application systems configured for use in a particular environment

Answer: d) Stand-alone application systems configured for use in a particular environment

Q34. According to the text, what are the five stages of the reuse-oriented (integration and configuration) process model?

- a) Specification; prototyping; refactoring; release; archiving
- b) Planning; coding; testing; debugging; deployment
- c) Design; marketing; sales; delivery; support
- d) Requirements specification; software discovery and evaluation; requirements refinement; application system configuration; component adaptation and integration

Answer: d) Requirements specification; software discovery and evaluation; requirements refinement; application system configuration; component adaptation and integration

Q35. What is an obvious advantage of reuse-oriented software engineering, according to the text?

- a) It eliminates the need for integration
- b) It guarantees the system perfectly meets every user need
- c) It removes the need for any requirements process
- d) It reduces the amount of software to be developed, reducing cost and risk, and usually leads to faster delivery

Answer: d) It reduces the amount of software to be developed, reducing cost and risk, and usually leads to faster delivery

Q36. What is a disadvantage of reuse-oriented software engineering mentioned in the text?

- a) It removes the need for software discovery and evaluation
- b) Requirements compromises are inevitable, and some control over system evolution is lost since reusable components are not controlled by the organization using them
- c) It cannot be used with web services
- d) It is always more expensive than writing everything from scratch

Answer: b) Requirements compromises are inevitable, and some control over system evolution is lost since reusable components are not controlled by the organization using them

Batch 3 – Questions 37 to 55: Section 2.2 Process Activities

Q37. According to Section 2.2, what are the four basic process activities common to software processes?

- a) Analysis, synthesis, compilation, and execution
- b) Planning, coding, marketing, and selling
- c) Design, testing, deployment, and retirement only
- d) Specification, development, validation, and evolution

Answer: d) Specification, development, validation, and evolution

Q38. How are these four basic activities organized in the waterfall model compared to incremental development?

- a) They are only used in the waterfall model
- b) They are identical in both models
- c) In the waterfall model they are organized in sequence, whereas in incremental development they are interleaved
- d) They are skipped entirely in incremental development

Answer: c) In the waterfall model they are organized in sequence, whereas in incremental development they are interleaved

Q39. Software specification (requirements engineering) is the process of:

- a) Only designing the user interface
- b) Understanding and defining what services are required from the system and identifying constraints on its operation and development
- c) Writing the final source code
- d) Testing the system after release

Answer: b) Understanding and defining what services are required from the system and identifying constraints on its operation and development

Q40. What is the purpose of a feasibility or marketing study, carried out before requirements engineering starts?

- a) To assess whether there is a need or market for the software and whether it is technically and financially realistic to develop
- b) To write the complete source code
- c) To train the future users of the system
- d) To test the final deployed system

Answer: a) To assess whether there is a need or market for the software and whether it is technically and financially realistic to develop

Q41. What are the three main activities in the requirements engineering process?

- a) Planning, marketing, and sales
- b) Coding, testing, and debugging
- c) Requirements elicitation and analysis, requirements specification, and requirements validation
- d) Design, implementation, and deployment

Answer: c) Requirements elicitation and analysis, requirements specification, and requirements validation

Q42. What is the difference between user requirements and system requirements, as described in the text?

- a) User requirements are abstract statements for the customer and end-user; system requirements are a more detailed description of the functionality
- b) They are exactly the same document
- c) System requirements are always shorter than user requirements
- d) User requirements are written only by programmers

Answer: a) User requirements are abstract statements for the customer and end-user; system requirements are a more detailed description of the functionality

Q43. What does requirements validation check for?

- a) Realism, consistency, and completeness of the requirements
- b) Only the programming language to be used
- c) Only the budget of the project
- d) Only spelling errors in the document

Answer: a) Realism, consistency, and completeness of the requirements

Q44. In agile methods, how is requirements specification treated?

- a) It is always a huge, separate document completed before any coding starts
- b) It is skipped entirely and never documented
- c) It is only done once at the very end of the project
- d) It is not a separate activity but part of system development, informally specified for each increment just before it is developed

Answer: d) It is not a separate activity but part of system development, informally specified for each increment just before it is developed

Q45. What is a software design, as defined in Section 2.2.2?

- a) Only the marketing plan for the product
- b) Only the final compiled binary
- c) Only the user manual
- d) A description of the structure of the software to be implemented, the data models and structures used, the interfaces between components and, sometimes, the algorithms used

Answer: d) A description of the structure of the software to be implemented, the data models and structures used, the interfaces between components and, sometimes, the algorithms used

Q46. According to Figure 2.5, which four activities may be part of the design process for information systems?

- a) Architectural design, database design, interface design, and component selection and design
- b) Hardware design, network design, training design, and support design
- c) Testing design, debugging design, deployment design, and retirement design
- d) Marketing design, sales design, pricing design, and branding design

Answer: a) Architectural design, database design, interface design, and component selection and design

Q47. What is architectural design concerned with?

- a) Only designing the database schema
- b) Identifying the overall structure of the system, its principal components, their relationships, and how they are distributed
- c) Only choosing a programming language
- d) Only writing unit tests

Answer: b) Identifying the overall structure of the system, its principal components, their relationships, and how they are distributed

Q48. Why must an interface specification be unambiguous?

- a) So that it never needs to be documented
- b) So it can be skipped during testing
- c) So that only one developer can ever use it
- d) So a component may be used by other components without them having to know how it is implemented

Answer: d) So a component may be used by other components without them having to know how it is implemented

Q49. What is the difference between testing and debugging, according to the text?

- a) Testing is only done by customers
- b) They are exactly the same activity
- c) Testing establishes the existence of defects, while debugging is concerned with locating and correcting these defects
- d) Debugging happens before any code is written

Answer: c) Testing establishes the existence of defects, while debugging is concerned with locating and correcting these defects

Q50. What is the goal of software validation (verification and validation, V&V)?

- a) To market the software product
- b) To show that a system both conforms to its specification and meets the expectations of the system customer
- c) To design the database schema
- d) To write new requirements for the next version

Answer: b) To show that a system both conforms to its specification and meets the expectations of the system customer

Q51. What are the three stages of the testing process shown in Figure 2.6?

- a) Design testing, documentation testing, and training testing
- b) Component testing, system testing, and customer testing
- c) Planning testing, coding testing, and marketing testing
- d) Unit testing, hardware testing, and network testing

Answer: b) Component testing, system testing, and customer testing

Q52. What is component testing?

- a) Testing performed only by the customer
- b) Testing individual components, such as functions or object classes, independently by the people developing the system
- c) Testing performed only after the product is released
- d) Testing the fully integrated system with real customer data

Answer: b) Testing individual components, such as functions or object classes, independently by the people developing the system

Q53. What is system testing concerned with, according to the text?

- a) Only testing a single isolated function
- b) Only checking spelling in the user manual
- c) Finding errors from unanticipated interactions between components and interface problems, and showing the system meets its requirements
- d) Only verifying the marketing plan

Answer: c) Finding errors from unanticipated interactions between components and interface problems, and showing the system meets its requirements

Q54. What is customer testing (also called beta testing for products)?

- a) Testing performed only by the original programmer
- b) A stage that occurs before any requirements are gathered
- c) The final stage of testing where the system is tested by the system customer or potential customers rather than with simulated test data
- d) A stage used only for embedded hardware systems

Answer: c) The final stage of testing where the system is tested by the system customer or potential customers rather than with simulated test data

Q55. What is the V-model of development, as mentioned in the text?

- a) A model with no relationship to testing
- b) A representation (used in plan-driven testing) showing validation activities that correspond to each stage of the waterfall process, where test plans link testing and development
- c) A model that replaces the waterfall model entirely
- d) A model used only for agile development

Answer: b) A representation (used in plan-driven testing) showing validation activities that correspond to each stage of the waterfall process, where test plans link testing and development

Essay Questions

Q56. Describe the waterfall model, including its five stages, and explain why it is appropriate for some types of systems but problematic for others.

Answer:

The waterfall model represents the fundamental software development activities as separate, sequential phases:

1. Requirements analysis and definition
2. System and software design
3. Implementation and unit testing
4. Integration and system testing
5. Operation and maintenance (normally the longest phase)

It is a plan-driven process: in principle, all activities are planned and scheduled before development starts, and each phase's result is one or more approved documents, with the next phase not starting until the previous one has finished.

Problems: in practice the software process is never purely linear — problems with requirements are found during design, design problems are found during coding, and so on, requiring feedback between phases. Both customers and developers may prematurely freeze the specification to avoid costly late changes, which can leave the system not doing what the user actually wants.

It is appropriate for: (1) embedded systems that must interface with inflexible hardware, (2) critical systems needing extensive safety/security analysis of complete specification and design documents, and (3) large systems developed by several partner companies that need complete specifications for independent subsystem development. It is not appropriate where informal team communication is possible and requirements change quickly — iterative development and agile methods suit those cases better.

Q57. Explain incremental development: how it works, its three advantages over the waterfall model, and the two problems it creates from a management perspective.

Answer:

How it works: an initial implementation is developed, feedback is gathered from users, and the software evolves through several increasingly functional versions. Specification, development, and validation are interleaved rather than separate. Each increment incorporates some needed functionality, usually with the most important/urgent functionality delivered earliest.

Three advantages over the waterfall model:

1. The cost of implementing requirements changes is reduced, since less analysis and documentation needs to be redone.
2. It is easier to get customer feedback, since customers can comment on working demonstrations rather than design documents.
3. Early delivery and deployment of useful software is possible, even before all functionality is included.

Two management-perspective problems:

1. The process is not very visible, since it is not cost-effective to produce documents reflecting every version, making it harder for managers to measure progress with regular deliverables.
2. System structure tends to degrade as new increments are added, becoming messier and more costly to extend — agile methods suggest regular refactoring to address this.

Q58. Describe the three main activities of the requirements engineering process (Section 2.2.1), and the four design activities shown in Figure 2.5 (Section 2.2.2).

Answer:

Requirements engineering process (three activities):

1. Requirements elicitation and analysis — deriving system requirements through observation of existing systems, discussions with users/procurers, and task analysis, often involving system models and prototypes.
2. Requirements specification — translating the gathered information into a document defining a set of requirements, at two levels: user requirements (abstract, for customers/end-users) and system requirements (a more detailed description of functionality).
3. Requirements validation — checking the requirements for realism, consistency, and completeness, and modifying the document to correct any discovered errors.

Design process activities (Figure 2.5, for information systems):

1. Architectural design — identifying the overall system structure, principal components/subsystems, their relationships, and distribution.
2. Database design — designing the system's data structures and how they will be represented in a database.
3. Interface design — defining unambiguous interfaces between system components so components can be used without knowing their implementation.
4. Component selection and design — searching for reusable components or designing new ones when none are suitable.

Q59. Explain the three stages of the testing process shown in Figure 2.6, and the difference between testing and debugging.

Answer:

Testing vs. debugging: testing establishes the existence of defects in a program; debugging is the separate process of locating and correcting those defects (generating hypotheses about observed behavior and testing them, often with the help of interactive debugging tools).

The three stages of testing:

1. Component testing — individual components (e.g., functions or object classes) are tested independently by the people developing the system, often with automated test tools (e.g., JUnit).
2. System testing — components are integrated into a complete system; this stage finds errors from unanticipated interactions between components and interface problems, and checks that functional and non-functional requirements are met.
3. Customer testing — the final stage before acceptance for operational use, where the system is tested with real customer data (for custom software) or through beta testing by selected users (for products).

End of Study Questions — Sections 2.1 and 2.2.